PREDICTING FRAME RATE FROM RENDERING LOAD

A Machine Learning Evaluation of FPS Prediction in Unity-Rendered Scenes

Constantinescu Theodor-Cristian 12601198

Dataset generated via a custom Unity script — controlled rendering-load sweeps with repeated FPS measurements per configuration.

MOTIVATION

Why predict FPS from rendering settings?

Frame rate is the practical bottleneck of real-time rendering

- Every real-time 3D application must keep FPS above a usability threshold — dropping below it breaks the experience.
- Rendering cost depends on several interacting factors: polygon count, light count, shadows, particle count, material complexity, and Unity's global quality preset.
- Manually profiling every combination of these settings is expensive; a predictive model lets developers estimate FPS for unseen configurations before ever rendering them.

This project asks: can we learn f : D → FPS from a systematic sweep of the input space, and if so, how accurately — and with which model family?

DATA

Dataset & Features

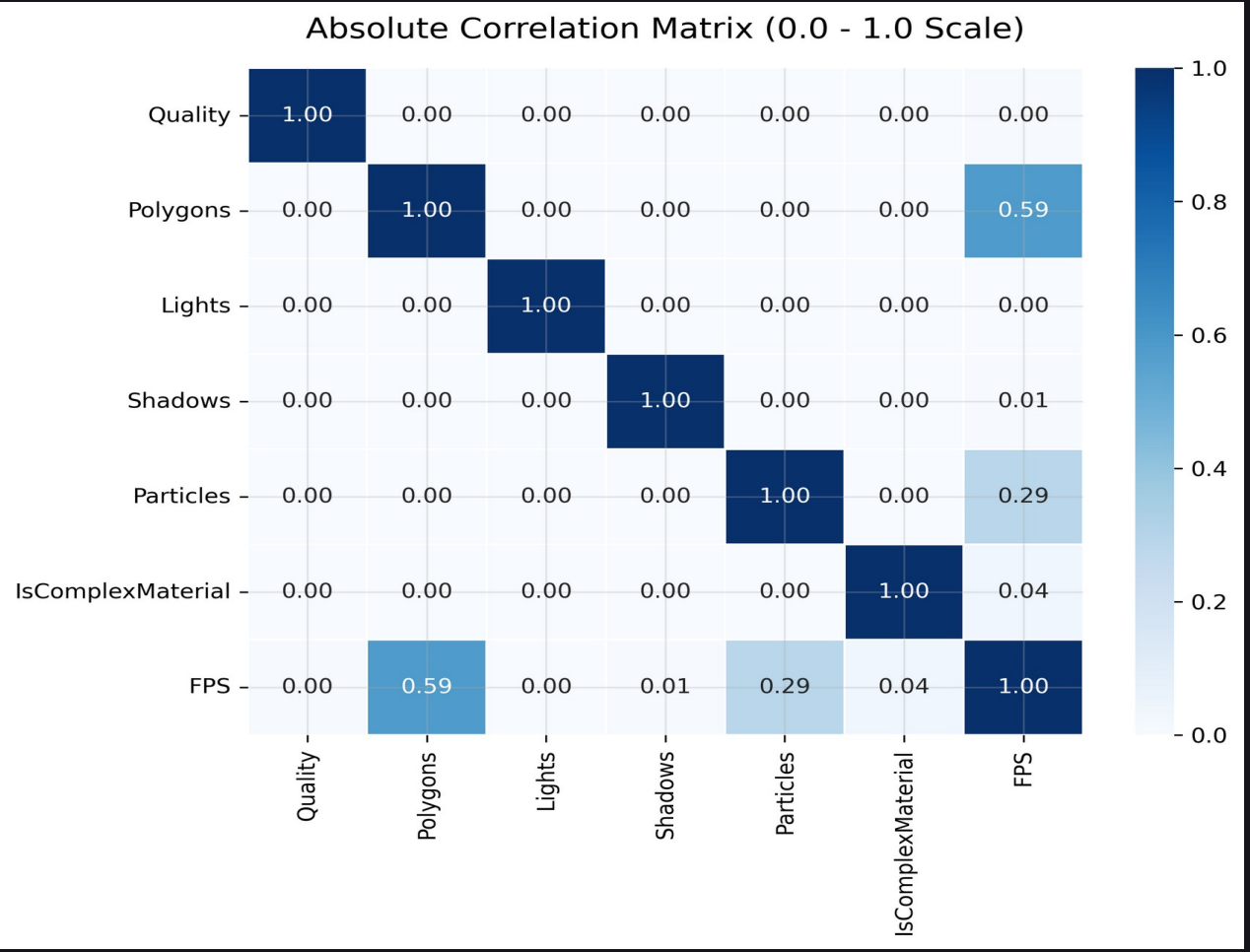
72,576	17,280	6	12,096
training rows	hidden test rows	input features	unique configurations

Feature	Type	Values in data
Quality	Categorical (6 levels)	0, 1, 2, 3, 4, 5 (Unity quality preset)
Polygons	Continuous / count	0 – 250,000
Lights	Continuous / count	0, 1, 2, 4, 8, 12, 16
Shadows	Boolean	0 / 1
Particles	Continuous / count	0 – 3,000
IsComplexMaterial	Boolean	0 / 1
FPS (target)	Continuous, positive	0.56 – 158.21 (train)

Random Factor

DATA

Data Characteristics: Correlation Matrix



Key observations

All the input features are linearly independent as the Unity Script measured the FPS independently on each configuration.

The value of the FPS seems to be influenced most by the polygon and particles count.

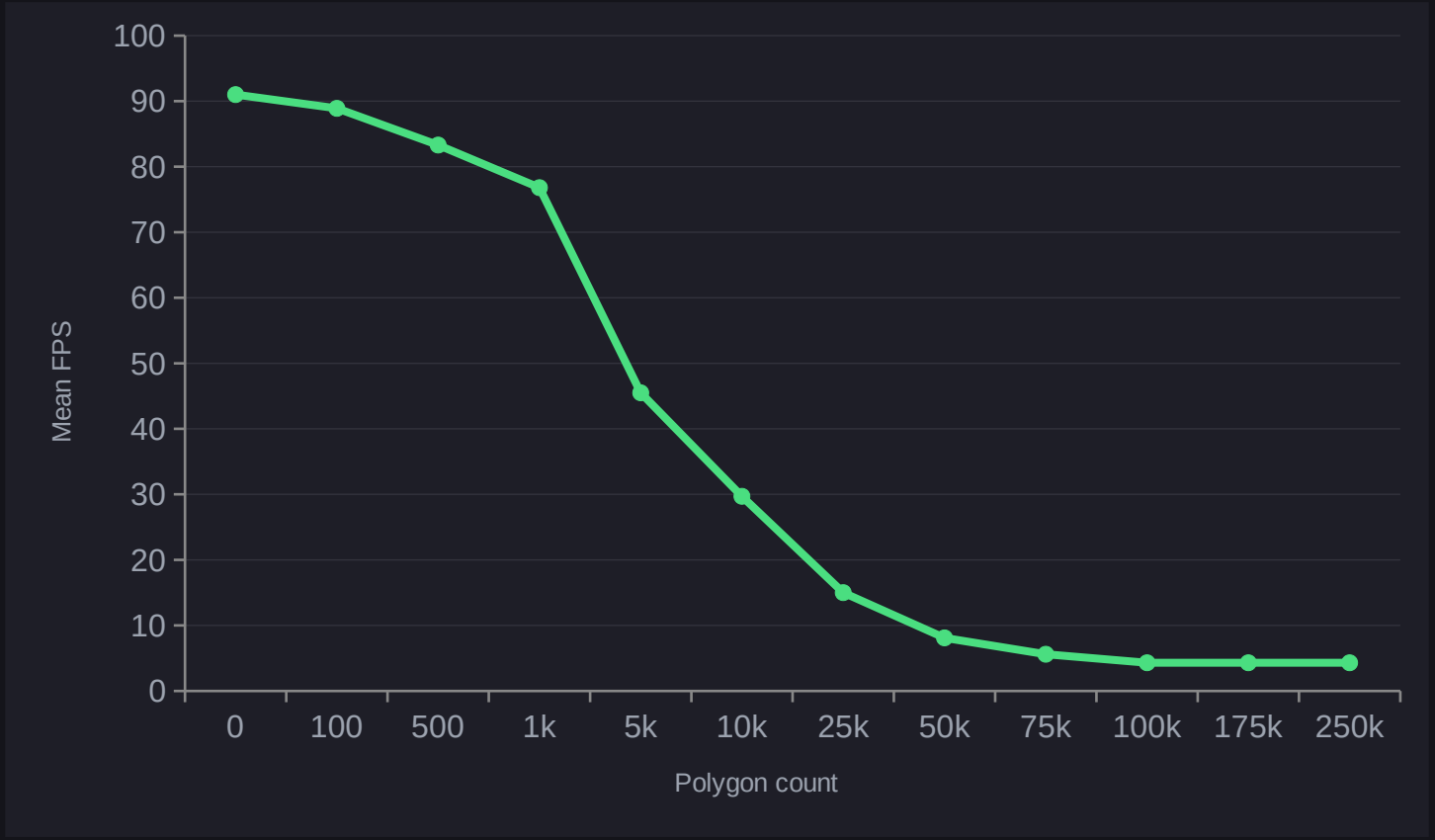
This linear independence is perfect for models such as Random Forest Regression.

The relationship between polygon count and FPS is strongly non-linear — near-flat at low load, then a steep asymptotic decay, flattening again near a hardware floor (~4–5 FPS).

Curve shown for Quality = 0, averaged across all other features.

DATA

Data Characteristics: FPS vs Polygon countx



Key observation

The relationship between polygon count and FPS is highly non-linear — near-flat at low values and with very small and near constant values at high polygon counts (~4–5 FPS).

This decay curve is the main reason a plain linear model struggles, and motivates the tree-based and polynomial model families evaluated later.

Curve shown for Quality = 0, averaged across all other features.

METHOD

Data Preparation

Held-out generalization test

No random 80/20 split is used.
Instead:

- Combinedx6.csv (72,576 rows) → entire training / CV po
- Hidden.csv (17,280 rows) → separate, untouched test set with different Polygon/Particle grid points
- Tests true generalization to unseen configurations — not just unseen noise on already-seen points

Grouped 5-fold cross-validation

Repeated rows of the same input point are assigned a shared group ID. GroupKFold guarantees all replicates of one configuration land entirely in train OR validation

same-color pairs = replicates of one point, split across the fold line
each color (= one input point) stays fully on one side of the fold line

each color (= one input point) stays fully on one side of the fold line

METHOD

Model & Evaluation Overview

Model	Role	Key preprocessing	Tuned via
Linear Regression	Base	StandardScaler	—
Random Forest	Advanced	None required	GridSearchCV (GroupKFold)
Decision Tree	Advanced	None required	GridSearchCV (GroupKFold)
Polynomial Ridge	Advanced	One-hot(Quality) + polynomial features + scaling	GridSearchCV (GroupKFold)

Design rationale

One base model (interpretable, minimal assumptions) is compared against three advanced models that can capture non-linear rendering-cost curves. All models are evaluated on the same untouched hidden set for a fair final comparison, regardless of which CV scheme tuned their hyperparameters.

METHOD — BASE MODEL

Linear Regression

Pipeline

StandardScaler → **PolynomialFeatures (degree = 2)** → **LinearRegression (OLS)**

Why lift features for the “base” model?

Since the dataset is not linear at all, I implemented a feature space lifting prior to fitting the model to bend into curves, in an attempt to offer better prediction and fitting to the model

Cross-validation

Evaluated with a manual 5-fold loop using the GroupKFold splitter defined earlier, then refit on the full training pool before predicting on the hidden set.

CV R^2 (train): 0.683

Hidden R^2: 0.429 | RMSE: 25.04 FPS

METHOD — ADVANCED MODEL 1

Random Forest Regressor

Multiple uncorrelated regression trees, averaged to reduce variance

Hyperparameter	Search grid
n_estimators	100, 300, 600
max_depth	10, 20, None
min_samples_leaf	1, 4, 16, 32, 64
max_features	sqrt, 1.0

Best parameters found

- n_estimators = 100

max_depth = 10

min_samples_leaf = 4

Tuning setup

GridSearchCV with cv = GroupKFold(5) and scoring = R^2 — the leakage-safe splitter, so both hyperparameter selection and its reported score reflect genuinely unseen configurations.

CV R^2: 0.989

Hidden R^2: 0.840 | RMSE: 13.25 FPS

METHOD — ADVANCED MODEL 2

Decision Tree Regressor

A single recursive-partition tree — the interpretable building block behind the forest

Hyperparameter	Search grid
criterion	squared_error, absolute_error
max_depth	5, 10, 15, 20, None
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4
max_features	None, sqrt, log2

Best parameters found

criterion = absolute_error
max_depth = 10
min_samples_split = 2
min_samples_le

GridSearchCV used cv = 5 (plain K-Fold), not GroupKFold — even though groups were passed to .fit(). Sklearn ignores groups for plain K-Fold, so replicate rows of the same input point could span train/validation during hyperparameter search (the run even prints a warning to this effect).

Hidden R^2: 0.824 | Hidden RMSE: 13.92 FPS | MAE: 8.26 FPS

METHOD — ADVANCED MODEL 3

Polynomial Ridge Regression

Higher-degree feature expansion with L2 regularization to control complexity

One-Hot (Quality) → Polynomial Features → StandardScaler → Ridge Regression

Hyperparameter	Search grid
poly degree	2, 3
interaction_only	True, False
ridge alpha	0.01, 0.1, 1.0, 10.0, 100.0

Best parameters found

degree = 3 | interaction_only = False | alpha = 0.01

Same cv = 5 (plain K-Fold) caveat as the Decision Tree applies here.

Hidden R^2: 0.234 | Hidden RMSE: 29.00 FPS | MAE: 17.47 FPS | MAPE: 273.3%

RESULTS

Results Overview

Model	Hidden Relative Error	Hidden R^2	Hidden RMSE (FPS)	Hidden MAE (FPS)
Linear Regression	247.78%	0.429	25.04	18.20
Random Forest	**48.29%**	**0.840**	**13.25**	**7.73**
Decision Tree	49.83%	0.824	13.92	8.26
Polynomial Ridge	**273.28%**	**0.234**	**29.00**	**17.47**

CV R^2 not shown for Decision Tree / Polynomial Ridge — their grid search used MSE-based scoring under a

Reading the table

B

A low RMSE combined with a high MAPE suggests the data contains many FPS values close to zero, which is true in many cases and the model is unable to minimize the residual error.

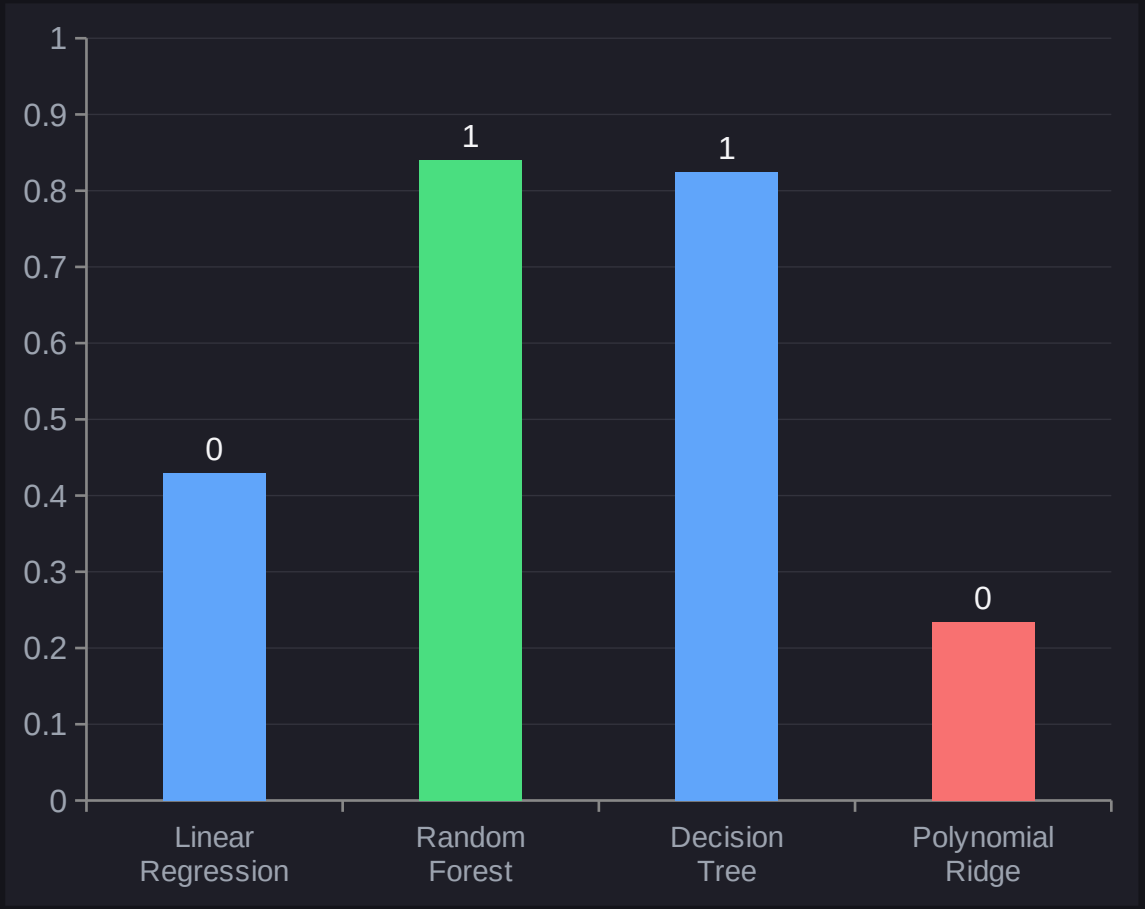
High RMSE and low MAE suggest there are some outliers in the dataset, more penalized by the RMSE.

d on the identical,
en test set — a fair basis
rison.

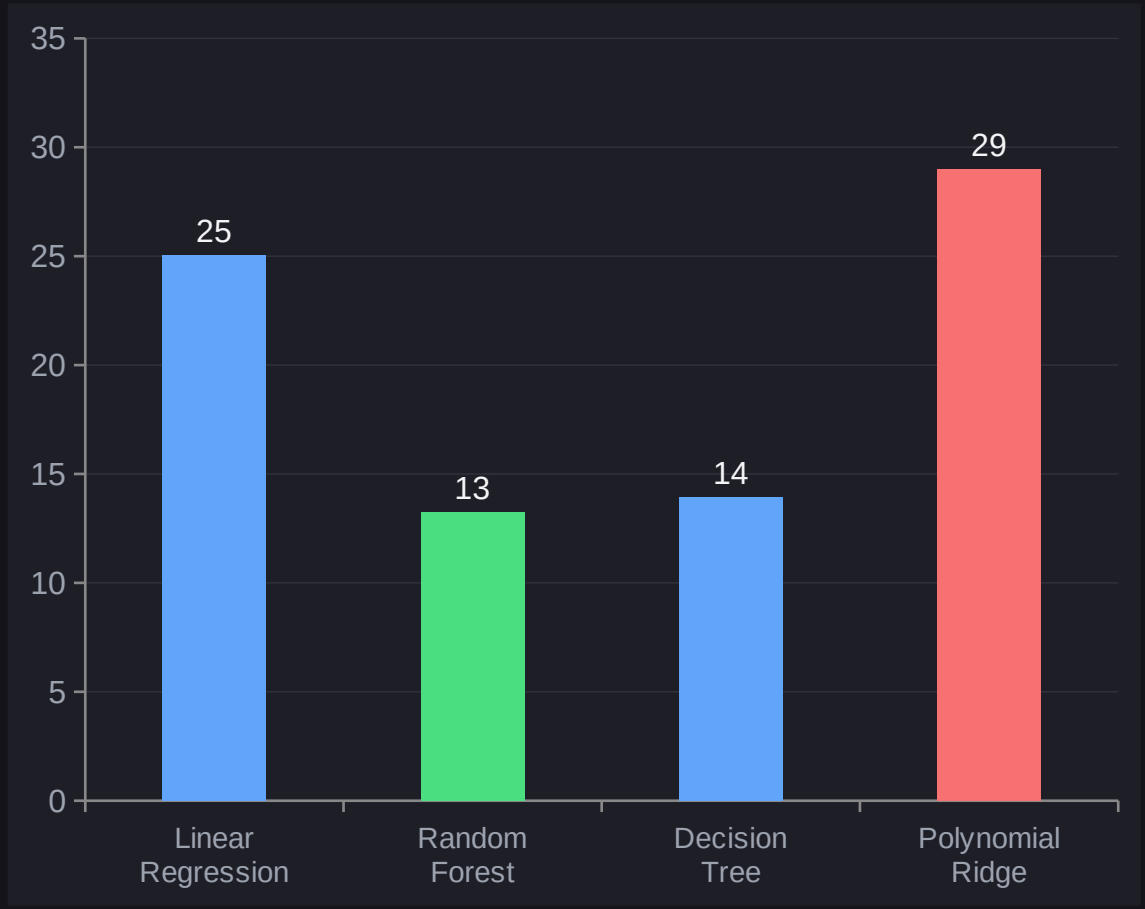
RESULTS

Model Comparison

Hidden R^2 (higher is better)



Hidden RMSE in FPS (lower is better)



DISCUSSION

Interpreting the Results

- **Non-linearity dominates.**

  The base linear model ($R^2 = 0.429$) confirms the FPS-vs-load curve seen in the data slide is not well captured by low-degree polynomial terms alone.

- **Random Forest generalizes best.**

  It has the largest R^2 (0.840) score with the role to “clean up” variance of the target function, achieving a higher score just by very little

- **Decision Tree is a strong single-model baseline.**

  The sharp boundaries of the dataset allows for the decision tree to accurately map a great part of the dataset

- **Polynomial Ridge underperforms even the base model.**

  Despite more flexibility (degree-3 interactions + regularization), Hidden $R^2 = 0.234$ is the worst result. Likely cause: polynomial blow-up at large Polygon values (up to 250,000) destabilizing coefficients, or overfitting to the training grid's exact structure.

SUMMARY

Random Forest best predicts FPS from rendering load

- A grouped train/hidden split and GroupKFold CV correctly account for repeated, noisy measurements of the same rendering configuration.
- Tree-based ensembles (Random Forest, $R^2 = 0.840$; Decision Tree, $R^2 = 0.824$) substantially outperform linear and polynomial regressors on this non-linear FPS surface.

  Feature-space expansion alone (Polynomial Ridge) does not guarantee better generalization — flexibility without the right inductive bias can perform worse than a simpler baseline.

- Feature-space expansion alone (Polynomial Ridge) does not guarantee better generalization — flexibility without the right induct